

SMC Fabric — SystemC Implementation Guide

SMC Fabric — SystemC Implementation Guide

1. Suggested Directory Layout
2. C++ Header Sketch
3. `b_transport` Implementation Patterns
 - 3.1 Internal-master sockets (MMIO, JTAG, Data-Accel, Log)
 - 3.2 `sys_axi_in` — inbound filter first
 - 3.3 `sep_axi_in` — truncate and route
4. Local Address Decode
5. DMI Policy
6. CMakeLists.txt Fragment
7. Integration in `smc_top`
8. Implementation Checklist

SMC Fabric — SystemC Implementation Guide

This document provides the C++ header sketch, implementation patterns, and integration checklist for `smc_fabric`. Follow the coding conventions established by `smc_cpu_cluster` (`smc/cpu_cluster/include/smc_cpu_cluster.h`).

1. Suggested Directory Layout

```
smc/smc_fabric/  
├─ doc/                ← this directory  
├─ include/  
│   └─ smc_fabric.h    ← SC_MODULE declaration  
├─ src/  
│   └─ smc_fabric.cpp   ← b_transport handlers, remap/filter logic  
├─ test/  
│   └─ fabric_tb.cpp    ← unit testbench  
└─ CMakeLists.txt
```

2. C++ Header Sketch

```
// =====
// include/smc_fabric.h
//
// PURPOSE
// -----
// SystemC TLM-2.0 LT model of the SMC Fabric (Router + Address Remap).
//
// Temporal decoupling: Bucket D (pure target). No quantum keeper.
// Every b_transport handler adds reg_access_ns to the incoming delay and
// forwards to the appropriate downstream socket.
// =====

#pragma once

#include <systemc.h>
#include <tlm.h>
#include <tlm_utils/simple_initiator_socket.h>
#include <tlm_utils/simple_target_socket.h>

#include <array>
#include <cstdint>
#include <string>

namespace smc {

class smc_axi_extension; // see cpu_cluster/include/smc_axi_extension.h

class smc_fabric : public sc_core::sc_module
{
public:
    // -----
    // Construction parameters
    // -----
    struct config {
        uint64_t local_base_addr = 0xC000'0000ULL; // LOCAL_BASE (RO register)
        uint64_t global_base_addr = 0x4000'0000ULL; // GLOBAL_BASE (RW register)
        uint64_t region_size      = 0x0200'0000ULL; // REGION_SIZE (RW register)

        // When true: M-mode/Xvisor output remap is bypassed
        bool no_addr_remap = true;

        unsigned num_inbound_filters = 16;
        unsigned num_outbound_filters = 16;

        double reg_access_ns = 1.0; // ns added to delay per register access
    };

    // -----
    // Target sockets – inbound masters → fabric
    // -----
    tlm_utils::simple_target_socket<smc_fabric, 64> jtag_axi_in {"jtag_axi_in"};
    tlm_utils::simple_target_socket<smc_fabric, 64> mmio_in {"mmio_in"};
    tlm_utils::simple_target_socket<smc_fabric, 64> data_accel_in {"data_accel_in"};
    tlm_utils::simple_target_socket<smc_fabric, 64> log_in {"log_in"};
    tlm_utils::simple_target_socket<smc_fabric, 64> sys_axi_in {"sys_axi_in"};
    tlm_utils::simple_target_socket<smc_fabric, 64> sep_axi_in {"sep_axi_in"};
};
```

```

// -----
// Initiator sockets – fabric → downstream targets
// -----
tlm_utils::simple_initiator_socket<smc_fabric, 64> to_front_port      {"to_front_port"};
tlm_utils::simple_initiator_socket<smc_fabric, 64> to_data_accel_ctrl {"to_data_accel_ctrl"};
tlm_utils::simple_initiator_socket<smc_fabric, 64> to_periph          {"to_periph"};
tlm_utils::simple_initiator_socket<smc_fabric, 64> to_dfd_apb         {"to_dfd_apb"};

tlm_utils::simple_initiator_socket<smc_fabric, 64> to_cpu_ctrl        {"to_cpu_ctrl"};
tlm_utils::simple_initiator_socket<smc_fabric, 64> to_aR_ctrl         {"to_aR_ctrl"};
tlm_utils::simple_initiator_socket<smc_fabric, 64> to_mR_ctrl         {"to_mR_ctrl"};
tlm_utils::simple_initiator_socket<smc_fabric, 64> to_xR_ctrl         {"to_xR_ctrl"};
tlm_utils::simple_initiator_socket<smc_fabric, 64> to_inbound_filter_ctrl {"to_inbound_filter_ctrl"};
tlm_utils::simple_initiator_socket<smc_fabric, 64> to_outbound_filter_ctrl {"to_outbound_filter_ctrl"};
tlm_utils::simple_initiator_socket<smc_fabric, 64> to_mailbox         {"to_mailbox"};
tlm_utils::simple_initiator_socket<smc_fabric, 64> to_dft_csr         {"to_dft_csr"};

tlm_utils::simple_initiator_socket<smc_fabric, 64> output_axi        {"output_axi"};

// -----
// Signals
// -----
sc_core::sc_in<bool> rst_n_i{"rst_n_i"};

// -----
// Construction
// -----
SC_HAS_PROCESS(smc_fabric);
explicit smc_fabric(sc_core::sc_module_name name, const config& cfg = {});

// -----
// Testbench / debug API
// -----
uint64_t read_global_base() const { return cfg.global_base_addr; }
uint64_t read_region_size() const { return cfg.region_size; }
void write_global_base(uint64_t v);
void write_region_size(uint64_t v);

struct alias_region {
    uint64_t start = 0;
    uint64_t end   = 0;
    int64_t  offset = 0;
    bool     cacheable = false;
    bool     valid     = false;
};
alias_region get_alias_region(unsigned n) const { return alias_regions[n]; }

private:
// -----
// b_transport callbacks
// -----
void bt_jtag      (tlm::tlm_generic_payload&, sc_core::sc_time&);
void bt_mmio      (tlm::tlm_generic_payload&, sc_core::sc_time&);
void bt_data_accel(tlm::tlm_generic_payload&, sc_core::sc_time&);
void bt_log       (tlm::tlm_generic_payload&, sc_core::sc_time&);
void bt_sys_axi   (tlm::tlm_generic_payload&, sc_core::sc_time&);
void bt_sep_axi   (tlm::tlm_generic_payload&, sc_core::sc_time&);

```

```

// -----
// Routing helpers
// -----
uint64_t apply_alias_remap(uint64_t addr) const;
bool      is_local(uint64_t addr) const;
uint32_t to_local_addr(uint64_t addr) const;

void route_local  (tlm::tlm_generic_payload&, sc_core::sc_time&);
void route_outbound(tlm::tlm_generic_payload&, sc_core::sc_time&);

bool inbound_filter_allow (uint64_t addr, uint8_t src_id, bool ns) const;
bool outbound_filter_allow(uint64_t addr, uint8_t src_id, bool ns) const;

enum class outbound_path { plain, mmode, xvisor };
outbound_path classify_outbound(uint64_t addr) const;

uint64_t apply_output_remap(uint64_t addr,
                           const std::array<alias_region, 8>& table,
                           uint8_t src_id_override,
                           smc_axi_extension* ext) const;

tlm_utils::simple_initiator_socket<smc_fabric, 64>*
local_decode(uint32_t addr32);

static void fill_deny_response(tlm::tlm_generic_payload&);
void      invalidate_all_dmi();

// SC_METHOD: reset all tables to power-on defaults
void reset_proc();

// -----
// State
// -----
config cfg_;

std::array<alias_region, 8> alias_regions_{}; // ALIAS_REMAP[0..7]
std::array<alias_region, 8> mmode_regions_{};  // MMODE_REMAP[0..7]
std::array<alias_region, 8> xvisor_regions_{}; // XVISOR_REMAP[0..7]

struct filter_entry {
    uint64_t addr_lo  = 0;
    uint64_t addr_hi  = 0; // exclusive
    uint8_t  src_id   = 0;
    uint8_t  src_mask = 0;
    bool     ns_req   = false;
    bool     src_en   = false;
    bool     ns_en    = false;
    bool     allow    = false;
    bool     enabled   = false;
};
std::array<filter_entry, 16> inbound_filter_{};
std::array<filter_entry, 16> outbound_filter_{};
};

} // namespace smc

```

3. `b_transport` Implementation Patterns

3.1 Internal-master sockets (MMIO, JTAG, Data-Accel, Log)

```
void smc_fabric::bt_mmio(tlm::tlm_generic_payload& trans, sc_core::sc_time& delay)
{
    uint64_t addr = apply_alias_remap(trans.get_address());
    trans.set_address(addr);

    if (is_local(addr)) {
        trans.set_address(to_local_addr(addr));
        route_local(trans, delay);
    } else {
        route_outbound(trans, delay);
    }
}
```

3.2 `sys_axi_in` — inbound filter first

```
void smc_fabric::bt_sys_axi(tlm::tlm_generic_payload& trans, sc_core::sc_time& delay)
{
    auto* ext = trans.get_extension<smc_axi_extension>();
    uint8_t src_id = ext ? (ext->source_id & 0xFu) : 0u;
    bool ns = ext ? ((ext->prot >> 1) & 1u) != 0 : true;

    if (!inbound_filter_allow(trans.get_address(), src_id, ns)) {
        fill_deny_response(trans);
        delay += sc_core::sc_time(cfg_.reg_access_ns, sc_core::SC_NS);
        return;
    }

    trans.set_address(to_local_addr(trans.get_address()));
    route_local(trans, delay);
}
```

3.3 `sep_axi_in` — truncate and route

```
void smc_fabric::bt_sep_axi(tlm::tlm_generic_payload& trans, sc_core::sc_time& delay)
{
    trans.set_address(to_local_addr(trans.get_address()));
    route_local(trans, delay);
}
```

4. Local Address Decode

`route_local()` performs a static range decode on the 32-bit post-masked address to select the downstream socket. Ranges are verbatim from `meta/crossbars/smc_local_xbar_pkg.sv` and `smc_internal_axi_lite_xbar_pkg.sv` (absolute addresses; `LOCAL_BASE = 0xC000_0000`).

Absolute address range	Socket	Description
0xC000_0000 – 0xC000_0FFF	to_front_port	WDT / debug
0xC004_0000 – 0xC015_FFFF	to_front_port	Scratchpad memory (SPM)
0xC400_0000 – 0xC7FF_FFFF	to_front_port	PLIC (outside alias aperture)
0xC800_0000 – 0xC801_FFFF	to_front_port	CLINT / BEU (outside alias aperture)
0xC003_8000 – 0xC003_FFFF	to_data_accel_ctrl	DMA + memory-zeroer control
0xC000_2000 – 0xC000_E7FF	to_periph	Peripheral main (UART, I2C, GPIO, ...)
0xC040_0000 – 0xC07F_FFFF	to_periph	Peripheral extended
0xC016_0000 – 0xC025_FFFF	to_dfd_apb	DFD registers (APB)
0xC000_F800 – 0xC000_FFFF	to_dft_csr	DFT CSRs
0xC001_0000 – 0xC001_1FFF	internal CSR decode	CPU control + fabric global CSRs
0xC001_2000 – 0xC001_2FFF	to_aR_ctrl	Alias remap CSRs
0xC001_3000 – 0xC001_3FFF	to_mR_ctrl	M-mode remap CSRs
0xC001_4000 – 0xC001_4FFF	to_xR_ctrl	Xvisor remap CSRs
0xC001_5000 – 0xC001_5FFF	to_inbound_filter_ctrl	Inbound filter CSRs
0xC001_6000 – 0xC001_6FFF	to_outbound_filter_ctrl	Outbound filter CSRs
0xC001_8000 – 0xC003_7FFF	to_mailbox	Mailbox

PLIC and CLINT sit beyond the 32 MiB alias aperture, so internal masters reach them only when `REGION_SIZE` is enlarged; otherwise they resolve as outbound.

5. DMI Policy

`smc_fabric` never grants DMI.

```
bool smc_fabric::get_direct_mem_ptr(tlm::tlm_generic_payload&,
                                   tlm::tlm_dmi& dmi_data)
{
    dmi_data.allow_read_write();
    dmi_data.set_start_address(0);
    dmi_data.set_end_address(sc_dt::uint64(-1));
    return false;
}

void smc_fabric::invalidate_all_dmi()
{
    to_front_port.invalidate_direct_mem_ptr(0, sc_dt::uint64(-1));
    to_data_accel_ctrl.invalidate_direct_mem_ptr(0, sc_dt::uint64(-1));
    to_periph.invalidate_direct_mem_ptr(0, sc_dt::uint64(-1));
    output_axi.invalidate_direct_mem_ptr(0, sc_dt::uint64(-1));
    // repeat for all other initiator sockets
}
```

Call `invalidate_all_dmi()` after any write to `GLOBAL_BASE`, `REGION_SIZE`, or any remap/filter register.

6. CMakeLists.txt Fragment

```
add_library(smc_fabric STATIC
    src/smc_fabric.cpp
)

target_include_directories(smc_fabric
    PUBLIC ${CMAKE_CURRENT_SOURCE_DIR}/include
)

target_link_libraries(smc_fabric
    PUBLIC SystemC::systemc
)

set_target_properties(smc_fabric PROPERTIES
    CXX_STANDARD          20
    CXX_STANDARD_REQUIRED ON
    CXX_EXTENSIONS        OFF
)
```

7. Integration in `smc_top`

```
smc_fabric::config fab_cfg;
fab_cfg.local_base_addr = 0xC000'0000ULL;
fab_cfg.global_base_addr = 0x4000'0000ULL;
fab_cfg.region_size      = 0x0200'0000ULL;
fab_cfg.no_addr_remap    = true;

smc_fabric fabric{"u_smc_fabric", fab_cfg};

// Inbound masters
cpu.mmio.bind(fabric.mmio_in);
dma.data_socket.bind(fabric.data_accel_in);
log_engine.log_socket.bind(fabric.log_in);
debug_module.jtag2axi_out.bind(fabric.jtag_axi_in);

// Local targets
fabric.to_front_port.bind(sram.socket);
fabric.to_periph.bind(periph_xbar.upstream);
fabric.to_data_accel_ctrl.bind(dma.ctrl_socket);

// NoC output
fabric.output_axi.bind(noc_stub.upstream);

// Reset
fabric.rst_n_i.bind(rst_primary_smc_n);
```

8. Implementation Checklist

- ☐ `smc_fabric.h` — all sockets, `config` struct, private helpers declared
- ☐ `smc_fabric.cpp` — all `b_transport` callbacks registered in constructor
- ☐ `bt_jtag` / `bt_mmio` / `bt_data_accel` / `bt_log` — alias remap → local/outbound split
- ☐ `bt_sys_axi` — inbound filter → local route
- ☐ `bt_sep_axi` — addr truncation → local route
- ☐ `apply_alias_remap()` — 8-entry table walk
- ☐ `is_local()` — dual-aperture check (local + global alias)
- ☐ `to_local_addr()` — 32-bit truncation with upper-7-bit masking
- ☐ `route_local()` — address-range decode → socket forward
- ☐ `route_outbound()` — `classify_outbound()` → optional remap → `outbound_filter_allow()` → `output_axi`
- ☐ `inbound_filter_allow()` / `outbound_filter_allow()` — 16-entry policy tables
- ☐ `fill_deny_response()` — `TLM_ADDRESS_ERROR_RESPONSE` + `0xBADCAB1E`
- ☐ CSR write handlers (`to_aR_ctrl`, `to_mR_ctrl`, `to_xR_ctrl`, filter sockets) — update tables + `invalidate_all_dmi()`
- ☐ `reset_proc()` — `SC_METHOD`, sensitive `rst_n_i.negedge_event()` — clear all tables
- ☐ `get_direct_mem_ptr()` — always returns `false`
- ☐ Unit test covering: local route, outbound route, filter deny, alias remap hit/miss, reset